HTML & CSS

Zero to Expert — Complete Guide

Phase-wise · Deep Explanations · Real-world Examples · Interview Q&A

■ PHASE 1	■ PHASE 2	■ PHASE 3
Complete Beginner What is HTML? · Tags Attributes · Classes · IDs Page Structure · First CSS Common CSS Properties	*Intermediate Box Model · Display Positioning · Flexbox CSS Grid · Pseudo-classes Responsive Design*	*Expert CSS Variables · Specificity Animations · Architecture Performance · Dark Mode Container Queries*

PHASE 1

Complete Beginner

What is HTML · Tags · Attributes · Classes · IDs · First CSS

1 What Is HTML?

HTML stands for **HyperText Markup Language**. It is the language used to describe the structure and content of every webpage you have ever visited. Every heading, paragraph, image, button, and link on the web is HTML.

HTML is **NOT a programming language**. You cannot do math, make decisions, or write logic in HTML. It simply describes what things ARE — this is a heading, this is a paragraph, this is an image. Think of it as the **skeleton** of a webpage. CSS adds skin and styling. JavaScript adds movement and behavior.

> ■ REAL WORLD
> Real World: When you open any website, your browser downloads an HTML file and renders it. Open Chrome DevTools (F12) > Elements tab on any site — that entire tree you see is HTML. Every company's website starts here.

```
<!-- Without HTML: just raw text — browser has no idea what's what -->
My Website
Welcome to my page
I am a Full Stack Developer

<!-- With HTML: browser now understands every piece of content -->
<!DOCTYPE html>
<html lang='en'>
<head>
<title>My Website</title>
</head>
<body>
<h1>Welcome to my page</h1>
<p>I am a Full Stack Developer</p>
<a href='/contact'>Contact me</a>
</body>
</html>

<!-- The browser reads this top to bottom and renders it visually -->
```

Q: What is HTML and what is it used for?

A: HTML (HyperText Markup Language) is the standard language for creating web pages. It describes the structure and content of a page using elements called tags. It is not a programming language — it cannot perform logic or calculations. It works alongside CSS (styling) and JavaScript (behavior) to create complete websites.

Q: What does HyperText mean in HTML?

A: HyperText refers to text that links to other text — hyperlinks. This is the core concept of the web: documents that link to other documents. The <a> (anchor) tag creates these hyperlinks. Markup refers to the system of annotating text with tags to give it meaning and structure.

2 What Are Tags?

Tags are the **building blocks of HTML**. A tag is a keyword wrapped in angle brackets < >. Tags tell the browser what a piece of content IS — a heading, a paragraph, a link, an image.

Almost every tag comes in pairs: an **opening tag** <h1> and a **closing tag** </h1> (notice the slash). Everything between them is the content. Think of tags like parentheses — they wrap around what they belong to.

```
<!-- ANATOMY OF A TAG -->
<p>This is a paragraph.</p>
^ ^
Opening tag Closing tag (has a /)

<!-- HEADING TAGS: h1 is largest, h6 is smallest -->
<h1>Main page title — use only ONE per page</h1>
<h2>Section heading</h2>
<h3>Sub-section heading</h3>
<h4>Smaller heading</h4>
<h5>Even smaller</h5>
<h6>Smallest heading</h6>

<!-- PARAGRAPH -->
<p>A block of text. Browsers add space above and below automatically.</p>

<!-- BOLD and ITALIC -->
<strong>Bold and semantically important</strong>
<em>Italic and semantically emphasized</em>
<b>Just visually bold — no semantic meaning</b>
<i>Just visually italic — no semantic meaning</i>

<!-- LINKS -->
<a href='https://google.com'>Click to visit Google</a>
<a href='/about'>Go to About page (same site)</a>
<a href='#section2'>Jump to section2 on this page</a>

<!-- LISTS -->
<ul> <!-- Unordered list (bullet points) -->
<li>First item</li>
<li>Second item</li>
</ul>

<ol> <!-- Ordered list (numbered) -->
<li>Step one</li>
<li>Step two</li>
</ol>

<!-- SELF-CLOSING TAGS: no content, no closing tag -->
<img src='photo.jpg' alt='My photo' /> <!-- Image -->
<input type='text' /> <!-- Text input -->
<br /> <!-- Line break -->
<hr /> <!-- Horizontal divider line -->
```

■ GOTCHA

Never skip closing tags (except self-closing ones like <img>,
, <input>). A missing or can break your entire page layout in unexpected ways.

Tag	What it creates	Self-closing?
<h1> to <h6>	Headings (h1 = largest, most important)	No
	Paragraph of text	No

<a>	Clickable link (needs href attribute)	No
/	Unordered / Ordered list container	No
	List item (goes inside ul or ol)	No
	Generic block container (no semantic meaning)	No
	Generic inline container (no semantic meaning)	No
<img>	Image (needs src and alt attributes)	Yes
<input>	Form input field	Yes
 	Line break	Yes

Q: What is the difference between block-level and inline elements?

A: Block-level elements (div, p, h1, section) take up the full width available and always start on a new line. Inline elements (span, a, strong, em) only take up as much width as their content and flow within a line of text. You can change this behavior with CSS display property.

Q: What is the difference between and <b>, and and <i>?

A: and carry semantic meaning — screen readers emphasize them, search engines treat them as important. <b> and <i> are purely visual — they look bold/italic but carry no special meaning to machines. Always prefer and in real projects.

3 What Are Attributes?

Tags tell the browser WHAT something is. But often we need extra information — WHICH image to show, WHERE a link goes, WHAT TYPE of input it is. This extra information is called an **attribute**.

Attributes live **inside the opening tag** and always follow this pattern: **attribute-name="value"**. Think of attributes as settings or options for a tag.

```
<!-- SYNTAX: <tagname attribute='value'>content</tagname> -->

<!-- Without href: a link with no destination — useless -->
<a>Click me</a>

<!-- With href attribute: now the browser knows where to go -->
<a href='https://google.com'>Click me</a>
^^^ ^^^^^^^^^^^^^^^^^^
name value

<!-- IMAGE: needs src (where is the file?) and alt (description) -->
<img
src='profile.jpg' <!-- source: path to the image file -->
alt='My profile photo' <!-- shown if image fails, read by screen readers -->
width='300' <!-- display width in pixels -->
height='300' <!-- display height in pixels -->
/>

<!-- INPUT: type attribute changes what kind of input it is -->
<input type='text' /> <!-- Regular text box -->
<input type='password' /> <!-- Text is hidden (dots) -->
<input type='email' /> <!-- Validates @ symbol -->
<input type='number' /> <!-- Numbers only -->
<input type='checkbox' /> <!-- Tick box -->

<!-- Input with multiple attributes -->
<input
type='text'
placeholder='Enter your name' <!-- ghost text before typing -->
required <!-- form won't submit if empty -->
disabled <!-- user cannot interact with it -->
/>

<!-- LINK tag: used in <head> to link CSS file -->
<link rel='stylesheet' href='style.css' />

<!-- META: page information for browsers and search engines -->
<meta charset='UTF-8' />
<meta name='description' content='My portfolio website' />
```

Attribute	Used On	What it does
href	<a>	URL the link goes to
src	<img>, <script>	Path/URL to the file
alt	<img>	Text description of image (accessibility + SEO)
type	<input>, <button>	What kind of input/button it is

placeholder	<input>	Ghost hint text shown before user types
required	<input>	Form cannot be submitted if this is empty
disabled	<input>, <button>	User cannot interact with this element
id	Any element	Unique identifier for the element on the page
class	Any element	Label for styling groups of elements with CSS
style	Any element	Inline CSS (avoid — use classes instead)

Q: What is the difference between the src and href attributes?

A: src (source) is used when the browser needs to EMBED an external resource directly into the page — like an image (<img src>) or script (<script src>). The browser fetches and renders that resource inline. href (hypertext reference) creates a LINK or relationship to another resource — like <a href> for navigation or <link href> for stylesheets. Clicking or loading the page triggers the connection.

Q: Why is the alt attribute on images important?

A: Three reasons: (1) Accessibility — screen readers read alt text to describe images to visually impaired users. (2) SEO — search engines cannot see images, so they read alt text to understand what the image is about. (3) Fallback — if the image fails to load, the alt text is displayed instead. Always write descriptive alt text. Use alt='' only for purely decorative images.

4 What Are Classes?

Imagine you have 50 students. You want to give all students in Group A a red badge. You don't visit each student individually — you say 'everyone in Group A gets a red badge.' **Classes in HTML work exactly like this.**

A class is a **reusable label** you put on HTML elements. In CSS, you then say: 'every element with this label should look like this.' **Many elements can share the same class.** One element can also have multiple classes at the same time.

```
<!-- Assigning a class to elements -->
<p class='highlight'>This paragraph is highlighted</p>
<p>This paragraph is normal</p>
<p class='highlight'>This one is also highlighted</p>
<!-- Both 'highlight' paragraphs get the same CSS styling -->

<!-- REUSE: same class on different tag types -->
<h2 class='highlight'>Headings can have it too</h2>
<span class='highlight'>And inline elements</span>

<!-- MULTIPLE CLASSES: space-separated, element gets all their styles -->
<div class='card featured large'>
This div has THREE classes: card, featured, AND large
It gets styling from all three
</div>

<!-- REAL EXAMPLE: buttons with shared + unique styles -->
<button class='btn btn--primary'>Save</button>
<button class='btn btn--secondary'>Cancel</button>
<button class='btn btn--danger'>Delete</button>
<!-- All share 'btn' styles (padding, font, border-radius) -->
<!-- Each has its own color from the second class -->
```

```
/* In CSS: target a class using a DOT before the name */

.highlight {
background-color: yellow;
font-weight: bold;
}
/* Applies to EVERY element with class='highlight' */

.card {
border: 1px solid #e2e8f0;
border-radius: 8px;
padding: 16px;
background: white;
}

.btn {
padding: 10px 20px;
border-radius: 6px;
font-size: 14px;
cursor: pointer;
}
.btn--primary { background: #6366f1; color: white; }
.btn--secondary { background: #f1f5f9; color: #0f172a; }
.btn--danger { background: #dc2626; color: white; }
```

Q: Can multiple elements share the same class?

A: Yes — that is the entire purpose of classes. You define a style once and apply that class to as many elements as you need. If you later change the CSS for that class, every element with it updates automatically. This is called DRY (Don't Repeat Yourself) — a core principle in software development.

Q: Can one element have multiple classes?

A: Yes. Separate them with spaces in the class attribute: class='card featured large'. The element receives all CSS rules from all three classes. If two classes set the same property, the one that appears LATER in the CSS file wins (cascade order). This is how component-based styling works — a base class for shared styles, modifier classes for variations.

5 What Are IDs?

If classes are group labels, **IDs are like personal names — unique identifiers**. An ID must appear only ONCE on a page. No two elements should ever share the same ID. Think of it like an Aadhaar number — every person has their own unique one.

```
<!-- Assigning IDs -->
<header id='main-header'>...</header>
<main id='page-content'>...</main>
<footer id='main-footer'>...</footer>

<!-- WRONG: duplicate IDs -- never do this -->
<div id='card'>First</div>
<div id='card'>Second</div> <!-- INVALID: duplicate -->

<!-- CORRECT: use class for repeating elements -->
<div class='card'>First</div>
<div class='card'>Second</div> <!-- Both can share a class -->

/* ■■■ In CSS: target ID using # before the name ■■■ */
```

```
#main-header { background: navy; color: white; padding: 20px; }
#page-content { max-width: 1200px; margin: 0 auto; }

<!-- ■■■ 3 Main Uses of ID ■■■ -->

<!-- 1. Anchor links: jump to a section on the page -->
<a href='#about'>Jump to About section</a>
...
<section id='about'>
<h2>About Me</h2>
</section>

<!-- 2. Connecting label to input (accessibility) -->
<label for='email-input'>Your Email</label>
<input id='email-input' type='email' />
<!-- Clicking the label now focuses the input field -->

<!-- 3. JavaScript targeting -->
<div id='error-box'>Something went wrong!</div>

// In JavaScript:
const box = document.getElementById('error-box');
box.textContent = 'Updated error message';
```

	Class	ID
HTML syntax	class="card"	id="main-header"
CSS syntax	.card { }	#main-header { }
Can repeat on page?	Yes — unlimited	No — must be unique
Multiple per element?	Yes — class='a b c'	Only one ID per element
Primary use	Styling groups of elements	Anchor links, JS targeting
CSS specificity weight	Lower (easier to override)	Higher (harder to override)
Best practice	Use for 95%+ of your styling	Avoid for styling — use for JS/anchors

Q: When should you use class vs ID?

A: Use class for styling — always. Classes are reusable, multiple elements can share them, and they have lower specificity making your CSS easier to maintain and override. Use ID for: (1) anchor links — <a href='#section'> paired with id='section', (2) connecting labels to inputs via the for attribute, (3) JavaScript targeting with getElementById. Avoid using IDs for CSS styling — they create specificity problems that are hard to debug.

6 Full HTML Page Structure — Every Line Explained

Every HTML page follows a required skeleton. Let's go through every single line so nothing is mysterious. This is the foundation of every webpage ever built.

```
<!DOCTYPE html>
<!--
Not a tag. A DECLARATION.
Tells the browser: 'This is a modern HTML5 document'
Always the absolute FIRST line. Always.
-->

<html lang='en'>
<!--
Root element: wraps EVERYTHING on the page.
lang='en' tells browsers and screen readers the language.
Helps with pronunciation, translation, and SEO.
-->

<head>
<!--
INVISIBLE to users. Contains page metadata (information ABOUT the page).
Think of it as the configuration/settings area.
-->

<meta charset='UTF-8' />
<!-- Character encoding. UTF-8 supports all characters: é, ñ, ■, ■ -->
<!-- Without this: special characters appear as garbled symbols -->

<meta name='viewport' content='width=device-width, initial-scale=1.0' />
<!-- CRITICAL for mobile. Without this: mobile shows zoomed-out desktop view -->
<!-- width=device-width: use actual screen width -->
<!-- initial-scale=1.0: don't zoom in or out by default -->

<title>Akshay Chavhan — Portfolio</title>
<!-- Text shown on browser tab AND in Google search results -->

<link rel='stylesheet' href='style.css' />
<!-- Links your CSS file. rel='stylesheet' tells browser what kind of file -->

</head>

<body>
<!--
Contains EVERYTHING the user sees and interacts with.
ALL visible content goes here.
-->

<header>
<a href='/' class='logo'>Akshay</a>
<nav>
<a href='/'>Home</a>
<a href='/about'>About</a>
<a href='/projects'>Projects</a>
</nav>
</header>
```

```
<main>
<section id='hero'>
<h1>Hi, I'm Akshay Chavhan</h1>
<p>Full Stack Developer based in Pune, India.</p>
<a href='/contact' class='btn btn--primary'>Hire Me</a>
</section>

<section id='projects'>
<h2>My Projects</h2>
<div class='card'>MSG2AI Platform</div>
<div class='card'>Project ADA</div>
</section>
</main>

<footer>
<p>&copy; 2024 Akshay Chavhan</p>
<!-- &copy; is an HTML entity for the copyright symbol: C-with-circle -->
</footer>

</body>
</html>
```

Q: What is the difference between <head> and <body>?

A: <head> contains metadata — information about the page that is NOT displayed to the user. This includes the page title, character encoding, links to CSS and JavaScript files, meta descriptions for SEO, and viewport settings for mobile. <body> contains everything the user actually sees and interacts with — all visible content: text, images, buttons, forms, navigation, and so on.

Q: What is the purpose of the viewport meta tag?

A: Without the viewport meta tag, mobile browsers render the page as if it were a 1024px wide desktop screen and then zoom out to fit, making everything tiny. width=device-width tells the browser to use the device's actual screen width. initial-scale=1.0 prevents any default zoom. This is required for any responsive design to work correctly on mobile devices.

7 What Is CSS and How Does It Work?

HTML gives structure. **CSS (Cascading Style Sheets) gives appearance**. CSS controls colors, fonts, sizes, spacing, layout — everything visual. The word **Cascading** means styles flow from parent elements to children and when two rules conflict, specific rules decide which wins.

CSS is written as **rules**. Each rule has: a **selector** (who to style), a **declaration block** { } (what to do), **properties** (what to change), and **values** (what to change it to).

```
/*
ANATOMY OF A CSS RULE:

selector {
property: value;
property: value;
}
*/

/* Tag selector: styles ALL h1 elements */
h1 {
color: #0f172a; /* text color */
```

```
font-size: 48px; /* size of text */
font-weight: bold; /* thickness of text */
margin-bottom: 16px; /* space below the element */
}

/* Class selector: styles elements with class='card' */
.card {
background-color: white;
border: 1px solid #e2e8f0;
border-radius: 8px; /* rounded corners */
padding: 24px; /* space INSIDE the element */
box-shadow: 0 2px 8px rgba(0,0,0,0.1); /* drop shadow */
}

/* Descendant: only <a> tags INSIDE <nav> */
nav a {
color: #6366f1;
text-decoration: none; /* removes the default underline */
font-weight: 500;
}

/* Multiple selectors: apply same rule to several elements */
h1, h2, h3 {
font-family: 'Inter', sans-serif;
}
```

3 Ways to Add CSS to HTML

```
<!-- 1. EXTERNAL FILE (Best practice — always use this) -->
<head>
<link rel='stylesheet' href='style.css' />
</head>
/* In style.css: h1 { color: blue; } */
/* Benefits: reusable across pages, clean separation, cacheable */

<!-- 2. INTERNAL STYLE BLOCK (OK for quick experiments) -->
<head>
<style>
h1 { color: blue; }
p { font-size: 16px; }
</style>
</head>
/* Problem: cannot reuse across pages */

<!-- 3. INLINE STYLES (Avoid — only use for dynamic JS-controlled values) -->
<h1 style='color: blue; font-size: 48px;'>Hello</h1>
/* Problems: hard to maintain, can't reuse, highest specificity */
/* Exception: when JavaScript dynamically sets a value at runtime */
```

Q: What does 'Cascading' mean in CSS?

A: Cascading means styles flow DOWN through the document — parent element styles are inherited by child elements unless overridden. The cascade also defines what happens when multiple rules target the same element: the rule with higher specificity wins, and if specificity is equal, the rule that appears later in the CSS file wins. This cascade is why CSS is both powerful and sometimes unpredictable.

Q: What is the difference between margin and padding?

A: Padding is space INSIDE the element — between the content and the element's border. It is part of the element's background. Margin is space OUTSIDE the element — between the element's border and surrounding elements. A simple way to remember: padding is a cushion inside a box, margin is the gap between boxes on a shelf.

8 The Most Common CSS Properties — A Complete Reference

These are the properties you will use in every project. Understanding these is the foundation of all CSS work.

```
/* ■■■ COLORS ■■■
```

```
border-radius: 50%; /* Perfect circle (square element) */
border-top: 3px solid blue; /* Only the top border */
outline: none; /* Remove focus outline (avoid!) */

/* ■■■ DISPLAY & VISIBILITY ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
display: none; /* Completely removed — takes NO space */
visibility: hidden; /* Invisible — still TAKES UP space */
opacity: 0; /* Transparent — still takes space + is clickable */
opacity: 0.5; /* 50% transparent */

/* ■■■ CURSOR ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
cursor: pointer; /* Hand cursor — for buttons and links */
cursor: not-allowed; /* Crossed circle — for disabled items */
cursor: text; /* Text I-beam cursor */

/* ■■■ BOX SHADOW ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
/* offset-x offset-y blur-radius spread-radius color */
box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
box-shadow: 0 4px 16px rgba(99, 102, 241, 0.3); /* colored shadow */

/* ■■■ OVERFLOW ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
overflow: hidden; /* Clips content that exceeds element bounds */
overflow: auto; /* Adds scrollbar only when needed */
overflow: scroll; /* Always shows scrollbar */
```

Property	Controls	Common Values
color	Text color	red, #6366f1, rgb(0,0,0)
background-color	Background color	white, #f8fafc, transparent
font-size	Text size	16px, 1rem, 1.5em
font-weight	Text thickness	normal, bold, 400, 700
font-family	Typeface	'Inter', Arial, sans-serif
line-height	Space between lines	1.6 (unitless recommended)
text-align	Horizontal text alignment	left, center, right
width / height	Element dimensions	300px, 100%, 50vw
max-width / min-width	Size limits	1200px, 200px
margin	Space OUTSIDE element	20px, 0 auto, 10px 20px
padding	Space INSIDE element	16px, 12px 24px
border	Element border	1px solid #e2e8f0
border-radius	Rounded corners	8px, 50%
display	How element is rendered	block, inline, flex, grid, none
opacity	Transparency	0 (invisible) to 1 (fully visible)
cursor	Mouse cursor appearance	pointer, not-allowed, text

Q: What is the difference between display:none, visibility:hidden, and opacity:0?

A: display:none completely removes the element from the page — it takes no space and cannot be interacted with. visibility:hidden makes the element invisible but it still occupies space in the layout — other elements do not move to fill its gap. opacity:0 makes the element fully transparent but it still occupies space AND remains interactive — you can still click it or focus on it even though invisible. Use display:none for toggling content. Use opacity for fade transitions.

Q: What is the difference between px, rem, and em units?

A: px (pixels) is an absolute unit — 16px is always 16px regardless of anything. rem (root em) is relative to the ROOT element's font size (typically 16px by default). 1rem = 16px, 1.5rem = 24px. rem is consistent because it always references the root. em is relative to the CURRENT element's font size — it compounds in nested elements which can cause unexpected results. Prefer rem for font sizes, px for borders/shadows.